

-ER

Alternatives, Clients, MVC

Yegor Bugayenko

Lecture #5 out of 8
80 minutes

The slidedeck was presented by the author in this [YouTube Video](#).

All visual and text materials presented in this slidedeck are either originally made by the author or taken from public Internet sources, such as web sites. Copyright belongs to their respected authors.

Examples and Alternatives

-Client Suffix

What About Performance?

Model-View-Controller (MVC)

Rultor + Takes



“When you need a manager, it’s often a sign that the managed are just plain old data structures and that the manager is the smart procedure doing the real work.”

— Carlo Pescio. *Your Coding Conventions Are Hurting You*.
<http://jttu.net/pescio2011your>. [Online; accessed 25-09-2024]. Apr.
2011

Chapter #1:

Examples and Alternatives

[[Parser](#) Reader Controller Validator Encoder Proof]

Parser

```

1 class Parser {
2     static int parseInt(String t) {
3         // Parse String into Integer
4     }
5     static float parseFloat(String t)
6     {
7         // Parse String into Float
8     }
9     // And many more methods...
10 }
11 int x = Parser.parseInt("42");

```

```

1 class StringAsInt implements Number
2 {
3     private final String txt;
4     StringAsInt(String t) { this.txt
5     = t; }
6     @Override int intValue() {
7         // Parse String into Integer
8         // and return the value
9     }
10 }
11 Number n = new StringAsInt("42");
12 int x = n.intValue();

```

[Parser [Reader](#) Controller Validator Encoder Proof]

Reader

```

1 class Reader {
2     static char readChar(InputStream
3         i) {
4         // Read the next char from the
5         // stream and return it, or
6         NULL
7         // if the stream is at the EOF
8     }
9 }
10
11 InputStream i = new FileInputStream
12     (...);
13 char c = Reader.readChar(i);

```

```

1 class Chars
2     private final InputStream is;
3     Chars(InputStream i)
4         this.is = i;
5     char next()
6         // Read the next char from the
7         // stream and throw exception
8         // if !exists()
9     bool exists()
10         // Return TRUE if not EOF
11     InputStream i = new FileInputStream
12         (...);
13     Chars chars = new Chars(i);
14     char c = chars.next();

```

[Parser [Reader](#) Controller Validator Encoder Proof]

[Parser Reader Controller Validator Encoder Proof]

Controller

```
1 class SimpleController {
2     @GET
3     @Path("/index")
4     HttpResponse index(HttpRequest e)
5     {
6         // Build an index page and
7         return
8     }
9     @POST
10    @Path("/update")
11    HttpResponse update(HttpRequest e
12    ) {
13    }
```

[Parser Reader Controller Validator Encoder Proof]

```
1 class IndexPage implements HttpPage
2   HttpResponse process(HttpRequest
3   e) {
4     // Build an index page and
5     return
6   }
7 class UpdatePage implements
8   HttpPage
9   HttpResponse process(HttpRequest
```

```
10 e) {
11   // Save new user information
12   // and return HTTP 303
13 }
14 new AllPages(
15   new IndexPage(),
16   new UpdatePage()
17 );
```

[Parser Reader Controller [Validator](#) Encoder Proof]

Validator

```

1 class Validator {
2     bool isValid(int age) {
3         return age >= 18;
4     }
5 }
6 int a = 23;
7 Validator v = new Validator();
8 if (!v.isValid(a)) {
9     throw new Exception(
10         "Age is not valid"
11     );
12 }

```

```

1 interface Age
2     int value();
3 class DefaultAge implements Age
4     private final int a;
5     DefaultAge(int a)
6         this.a = a;
7     @Override int value()
8         return this.a;
9 class ValidAge implements Age {
10     private final Age origin;
11     ValidAge(Age age)
12         this.origin = age;
13     @Override int value()
14         int v = this.origin.value();
15         if (v < 18)
16             throw new Exception("Age is not valid");
17         return v;
18 Age a = new ValidAge(new DefaultAge(23));

```

[Parser Reader Controller Validator [Encoder](#) Proof]

Encoder

```

1 package java.net;
2
3 class URLEncoder {
4     static String encode(String s, String enc) {
5         // Encode the string "s" using
6         // the "enc" encoding and return
7         // the encoded string
8     }
9 }
10 String e = URLEncoder.encode("@foo");
11 e.equals("%40foo");

```

You may want to read more about this in my blog [1, 3].

```

1 class Encoded implements String {
2     private final String origin;
3     private final String encoding;
4     Encoded(String s, String enc) {
5         this.origin = s;
6         this.enc = encoding;
7     }
8     @Override String value() {
9         // Encode the string "origin" using
10        // the "encoding" and return
11        // the encoded string
12    }
13 }
14 String e = new Encoded("@foo");
15 e.value().equals("%40foo");

```

[Parser Reader Controller Validator [Encoder](#) Proof]

The right snippet won't work in Java, since `String` is a final class, not an interface, unfortunately.

[Parser Reader Controller Validator Encoder [Proof](#)]

Proof: Classes with -ER Suffixes Are More Complex

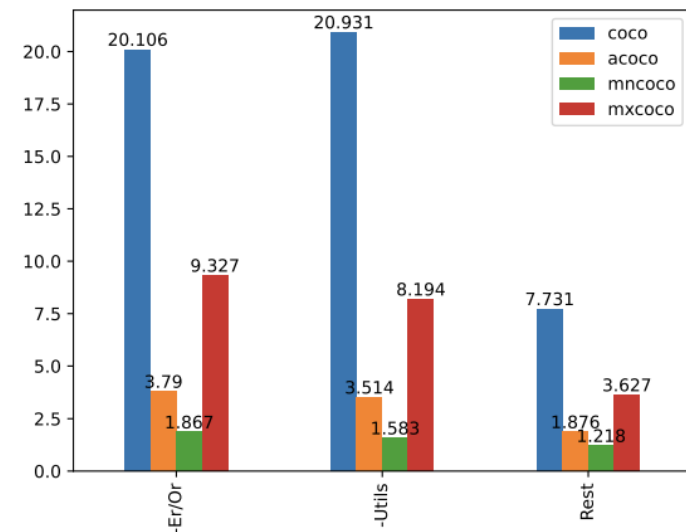


Figure 3. The average values of total CoCo for three groups of Java classes: the classes with “-Er/-Or” and “-Utils” suffixes are at least 2.5 times more complex.

“We took 13,861 Java classes from 212 open source repositories, divided them into three groups (‘-Er’ classes, ‘-Utils’, and others), and evaluated their complexity. Because average CC and CoCO in the first two groups were almost 3x larger than in the third group, we concluded that functor classes may be considered bad design.”

Source: Anna Sukhova et al. *Java Classes With “-Er” and “-Utils” Suffixes Have Higher Complexity*. 2024. arXiv: [2403.17430](#) [cs.PL]

Chapter #2:

-Client Suffix

[AWS]

AWS Java Client

```
1 class AmazonS3Client {
2     createBucket(String name);
3     deleteBucket(String name);
4     doesBucketExist(String name);
5     getBucketAcl(String name)
6     getBucketPolicy(String name);
7     listBuckets();
8     // 160+ more methods here
9 }
10 client = new AmazonS3Client("us-1");
11 client.createBucket("foo");
12 client.putObject("foo", "a.txt");
13 client.writeObject("foo", "a.txt", "data");
```

```
1 region = new S3Region("us-1");
2 bucket = region.createBucket("foo");
3 object = bucket.putObject("a.txt");
4 object.write("data");
```

It's here: [jcabi/jcabi-s3](https://github.com/jcabi/jcabi-s3).

The left snippet is: 1) procedural, 2) hard to test, 3) resembles a utility class, and 4) is hard to extend. The right one is object-oriented.

Chapter #3:

What About Performance?

[Sticky Safe]

Sticky Parseable Object

```

1 class StringAsInt implements Number
  {
2   private final String txt;
3   StringAsInt(String t) { this.txt
    = t; }
4   @Override int intValue() {
5       // Parse String into Integer
6       // and return the value
7   }
8 }
9
10 Number n = new StringAsInt("42");
11 int x = n.intValue();

```

```

1 class StickyInt implements Number {
2     private final Number origin;
3     private int cache = 0;
4     private bool cached = false;
5     StickyInt(Number n) { origin = n;
6     }
7     @Override int intValue() {
8         if (!cached) {
9             cache = origin.intValue();
10            cached = true;
11        }
12        return cache; } }

```

Alternatives -Client Performance MVC Takes

18/26

[Sticky Safe]

Is it thread-safe though?

[Sticky Safe]

Thread-safe Sticky Parseable Object

```

1 class StickyInt implements Number {
2     private final Number origin;
3     private int cache = 0;
4     private bool cached = false;
5     StickyInt(Number n) { origin = n;
6         }
7     @Override int intValue() {
8         if (!cached) {
9             cache = origin.intValue();
10        }
11        return cache;
12    }
13 }

```

```

1 class StickyInt implements Number {
2     private final Number origin;
3     private final AtomicReference<Integer> cache =
4         new AtomicReference<Integer>(null);
5     StickyInt(Number n) { origin = n; }
6     @Override int intValue() {
7         return cache.updateAndGet(
8             x -> {
9                 if (x == null) {
10                     return origin.intValue();
11                 }
12                 return x;
13             }
14         );
15     }
16 }

```

[Sticky [Safe](#)]

The left snippet is not thread-safety, while the right one is.

Chapter #4:

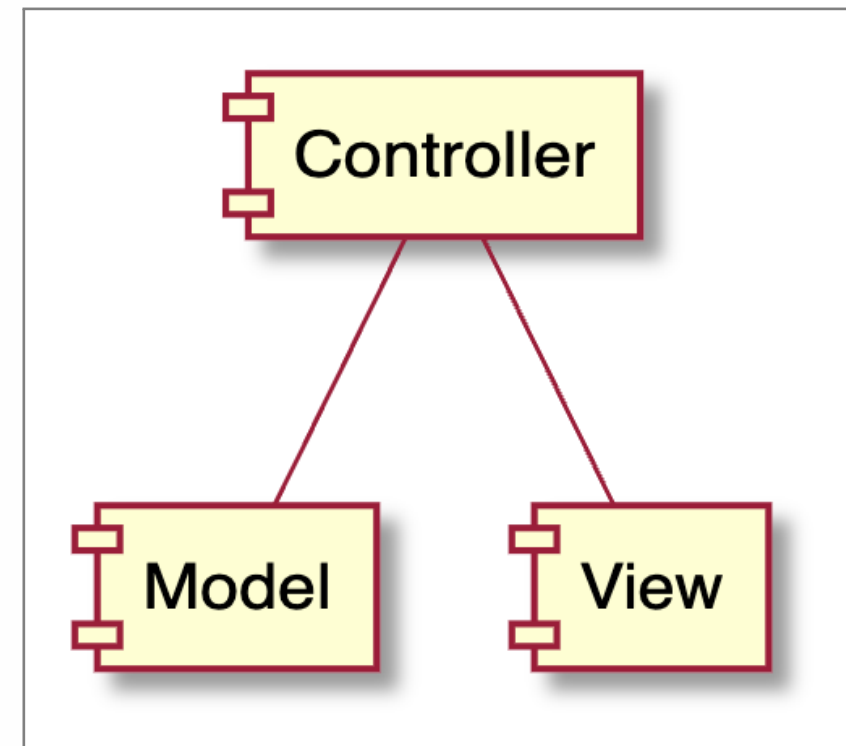
Model-View-Controller (MVC)

[[Controller](#) HTML]

The Controller

```

1 class Controller {
2     @GET
3     @Path("/b{id}")
4     String index(int id) {
5         Book book = em.findById(id);
6         View v = new HtmlView("book.
7         html");
8         v.set("title", book.getTitle())
9         ;
10        return v.renderHtml();
11    }
12 }
```



This is bad OOP [2].

[Controller [HTML](#)]

Book as HTML

```

1 class Controller {
2     @GET
3     @Path("/b{id}")
4     String index(int id) {
5         Book book = em.findById(id);
6         View v = new HtmlView("book.
html");
7         v.set("title", book.getTitle())
8         ;
9         return v.renderHtml();
10    }

```

```

1 interface Book
2     String title();
3 class PgBook implements Book
4     String title() // loads from PostgreSQL
5 interface Page
6     String html();
7 class HtmlBook implements Book, Page
8     String html() // renders in HTML
9     String title() // returns origin.title()
10 class PageOnPath implements Page
11     private final String path;
12     private final Page origin;
13     String html() // renders if path matches

```

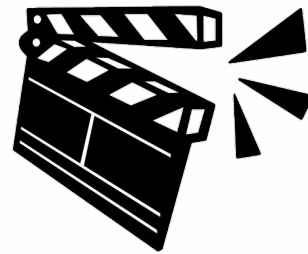
Check [yegor256/jpages](#) and [yegor256/takes](#).

Chapter #5:

Rultor + Takes



rultor.com



takes.org

```

return new TkWithHeaders(
    new TkVersioned(
        new TkGzip(
            new TkMeasured(
                new TkFlash(
                    new TkAppFallback(
                        new TkAppAuth(
                            new TkForward(
                                TkApp.regex(talks, pulse, toggles)
                            )
                        )
                    )
                )
            )
        )
    ),
    String.format("X-Rultor-Revision: %s", TkApp.REV),
    "Vary: Cookie"
);

```

```

private static Take regex(final Talks talks,
    final Pulse pulse, final Toggles toggles) {
    return new TkFork(
        new FkRegex("/robots.txt", ""),
        new FkRegex("/ticks", new TkTicks(pulse)),
        new FkRegex("/status", new TkStatus(pulse)),
        new FkRegex("/s/.*", new TkRedirect()),
        new FkRegex("/sitemap", new TkSitemap(talks)),
        new FkRegex(
            "/xsl/.*",
            new TkWithType(new TkClasspath(), "text/xsl")
        ),
        new FkRegex(
            "/js/.*",
            new TkWithType(new TkClasspath(), "text/javascript")
        ),
        new FkRegex(
            "/css/.*",
            new TkWithType(new TkClasspath(), "text/css")
        ),
        new FkRegex("/", new TkHome(talks, toggles)),
        new FkRegex("/b/([/a-zA-Z0-9_\\-\\.]+)", new TkButton()),
        new FkRegex("/t/([0-9]+)-([a-f0-9]+)", new TkDaemon(talks)),
        new FkRegex("/p/([/a-zA-Z0-9_\\-\\.]+)", new TkSiblings(talks)),
        new FkAdminOnly(
            new TkFork(
                new FkRegex("/t/([0-9]+)", new TkTalk(talks)),
                new FkRegex("/t/([0-9]+)/kill", new TkTalkKill(talks)),
                new FkRegex("/t/([0-9]+)/delete", new TkTalkDelete(talks)),
                new FkRegex("/toggles/read-only", new TkToggles(toggles))
            )
        )
    );
}

```

<https://github.com/yegor256/rultor/blob/master/src/main/java/com/rultor/web/TkApp.java>

References

- [1] Yegor Bugayenko. *Don't Create Objects That End With -ER*.
<https://www.yegor256.com/150309.html>.
[Online; accessed 22-09-2024]. Mar. 2015.
- [2] Yegor Bugayenko. *MVC vs. OOP*.
<https://www.yegor256.com/161213.html>.
[Online; accessed 22-09-2024]. Dec. 2016.
- [3] Yegor Bugayenko. *Yet Another Evil Suffix for Object Names: Client*.
<https://www.yegor256.com/170912.html>.
[Online; accessed 22-09-2024]. Sept. 2017.
- [4] Carlo Pescio. *Your Coding Conventions Are Hurting You*. <http://jttu.net/pescio2011your>. [Online; accessed 25-09-2024]. Apr. 2011.
- [5] Anna Sukhova et al. *Java Classes With “-Er” and “-Utils” Suffixes Have Higher Complexity*. 2024. arXiv: [2403.17430](https://arxiv.org/abs/2403.17430) [cs.PL].